

IBMEC
MBA EM INTELIGÊNCIA ARTIFICIAL
GENERATIVA - PCDF

Disciplina: RAG & CAG Aplicados a Direito e Segurança Pública

AVALIAÇÃO E OTIMIZAÇÃO DE
TÉCNICAS DE RECUPERAÇÃO EM
SISTEMAS RAG

**Um Estudo de Caso utilizando a Constituição da República Federativa
do Brasil de 1988**

Projeto base: Projeto Final (aula12/projeto_final) — API de Ingestão
Inteligente + RAG

Grupo 2

Rafael Henrique Soares Nogueira Rocha

Reginaldo Toshiaki Tanno

Renato Silva e Lira

Rodrigo Taboada Macedo

Rogério Alves da Conceição

Valdeci Cardoso da Mata Filho

Sumário

Relatório Técnico: Avaliação e Otimização de um Sistema RAG no Domínio Jurídico 3

1. Introdução e Construção do Dataset 3

2. Metodologia de Avaliação 3

3. Experimentos e Resultados 5

 3.1. Fase 0 (Baseline) e Fase 1 (Extração)..... 5

 3.2. Fase 2: Estratégias de Chunking..... 6

 3.3. Fase 3: Avaliação de Modelos de Vetorização (*Embedding*) 8

 3.4. Fase 4: Parâmetros de Recuperação e Busca Híbrida..... 8

 3.5. Fase 5: Expansão e Reescrita de Consulta (*Query Enhancement*) 9

 3.6. Fase 6: Reordenação Semântica (*Reranking*) 10

 3.7. Fase 7: Recuperação Baseada em Grafos (*LightRAG*) 11

 3.8. Fase 8: Avaliação de Geração (RAGAS) 11

4. Resultados Consolidados e Configuração Ótima 13

5. Discussão Crítica e Limitações 15

Relatório Técnico: Avaliação e Otimização de Técnicas de Recuperação em Sistemas RAG

1. Introdução e Construção do Dataset

Este relatório descreve a avaliação estruturada de um sistema de Geração Aumentada por Recuperação (RAG) aplicado a um *corpus* jurídico. O conjunto de documentos base foi construído a partir do texto completo da Constituição Federal, totalizando 274 artigos únicos. O Ato das Disposições Constitucionais Transitórias (ADCT) foi excluído para evitar colisões de identificadores, visto que sua numeração reinicia de forma independente.

Para o conjunto de avaliação foram elaboradas **22 perguntas (avaliacao/dataset.json)** divididas em três categorias: factuais (9 perguntas, mapeadas para 1 artigo), reformuláveis (**6 perguntas com vocabulário coloquial, focadas em testar técnicas de reescrita de consulta**) e **temáticas/multi-hop (7 perguntas exigindo a recuperação de 2 a 3 artigos)**. As perguntas foram intencionalmente redigidas em linguagem coloquial, evitando a cópia exata dos termos legais para mitigar o viés de sobreposição lexical.

O pareamento entre perguntas e artigos, necessário ao cálculo das métricas de recuperação, exigiu solução própria, pois o campo `id_original` referenciado em `app/consulta.py` não é populado em nenhuma etapa do projeto original — cada *chunk* recebe apenas **`meta={"arquivo": nome_do_arquivo}`** e, como a fonte é um único PDF, esse metadado não fornece a granularidade requerida pelo gabarito.

Optou-se, então, por identificar os artigos cobertos por cada fragmento (*chunk*) recuperado por meio de expressões regulares aplicadas ao próprio conteúdo do fragmento (**`(Art\.\?s*\d+[º])?(-[A-Z])?`**). A abordagem independe da técnica de particionamento e dispensa modificações no pipeline de indexação para acomodar a granularidade do gabarito.

2. Metodologia de Avaliação

O desempenho foi mensurado por quatro métricas padronizadas de recuperação de informação: **Hit@5, Recall@5, MRR — calculado sobre a lista completa de resultados — e NDCG@10**. Este último foi

computado em modo graduado, atribuindo-se relevância 2 aos artigos centrais à resposta e 1 aos meramente relevantes, com implementação em *avaliacao/comum.py* a partir *de sklearn.metrics.ndcg_score*.

Para evitar uma chamada ao modelo de linguagem por pergunta apenas para medir a recuperação, acrescentou-se à função *app/busca_avancada.construir()* o parâmetro **apenas_recuperacao = True**, que reconstrói o mesmo pipeline Haystack de cada técnica até a etapa de busca, suprimindo os nós de prompt e geração. Ressalte-se que as técnicas de reescrita de consulta (*multi_query*, *rag_fusion* e *step_back*) preservam sua própria chamada de reescrita ao modelo, por integrarem a técnica avaliada e não a geração da resposta final.

Cada experimento reindexou a base no OpenSearch a partir do zero, *avaliacao/reindexar.py* apaga o índice antes de cada indexação, assegurando comparação controlada: uma variável por experimento, mesmo conjunto de avaliação e mesmo **top_k** quando este não constituía a variável em teste.

Componentes e padrões arquiteturais foram reaproveitados sem modificação de implementações anteriores: o módulo **bench_embeddings/app**, para a comparação de modelos de *embedding* (Fase 3); o padrão de busca híbrida BM25+RRF (OpenSearchHybridRetriever); o padrão de reordenação por *cross-encoder* (TransformersSimilarityRanker com BAAI/bge-reranker-v2-m3); e a configuração de avaliação generativa RAGAS apoiada em modelo hospedado na Groq (ResponseRelevancy com strictness=1, dado que o provedor admite apenas n=1).

A observabilidade foi assegurada pelo LangFuse. A tentativa inicial de instanciá-lo pelo docker-compose de referência falhou, pois a imagem corrente corresponde à versão 3, que exige ClickHouse — ausente naquela composição, restrita ao Postgres.

A solução consistiu em clonar o repositório oficial da ferramenta, que já disponibiliza a composição completa da versão 3 (Postgres, ClickHouse, Redis, MinIO, além dos serviços *web* e *worker*), executada por meio do Podman.

Com projeto e chaves gerados na interface, o rastreamento operou de ponta a ponta: uma consulta de teste sob a melhor configuração (com reordenação) produziu um *trace* completo, com latência discriminada por componente, **4,59s no embedder, 0,07 s no retriever, 16,43 s no**

ranker (cross-encoder) e 1,94 s no modelo de linguagem (2.222 tokens de prompt e 245 de conclusão). O registro corrobora, com dados de execução real, o mesmo diagnóstico de custo observado nas métricas *offline* da Fase 6 (Seção 3.5): o *cross-encoder* é, com ampla margem, o componente mais oneroso do pipeline.

3. Experimentos e Resultados

3.1. Fase 0 (Baseline) e Fase 1 (Extração)

O experimento inicial (Fase 0) utilizou as configurações padrão: particionamento automático via Docling, vetorização via nomic-embed-text e busca densa no OpenSearch. Os resultados demonstraram desempenho significativamente insatisfatório, com **Hit@5 de 0.1818 e Recall@5 de 0.1364**.

A investigação dos logs revelou que o extrator de texto falhou silenciosamente a partir da página 82 devido a um erro de alocação de memória (std::bad_alloc). Como o truncamento manteve um volume parcial de texto, o mecanismo de *fallback* automático não foi acionado, resultando na extração de apenas 18% do conteúdo real. Na Fase 1, a substituição do **Docling** pelo **PyMuPDF** garantiu a extração integral dos **698.372** caracteres, elevando imediatamente o **Hit@5 para 0.4091 e o NDCG@10 para 0.3133**.

A partir desse diagnóstico, formulou-se a hipótese de que a deficiência na extração limitava severamente todas as etapas subsequentes (downstream) do pipeline. Para validar essa premissa (Fase 1), a extração foi substituída pela biblioteca PyMuPDF, garantindo a leitura integral dos 698.372 caracteres referentes aos 250 artigos.

Para isolar rigorosamente a variável de extração, foram executados dois novos experimentos, conforme detalhado na Tabela 1: o exp02, que além da nova extração aplicou um particionamento por janela de sentença; e o exp02b, que manteve o particionamento hierárquico idêntico ao da Fase 0.

Tabela 1: Comparativo de métricas de recuperação isolando o método de extração.

exp	Configuração	Hit@5	Recall@5	MRR	NDCG@10
exp01	chunking=auto(hierarquico via Docling truncado), embedding=nomic-embed-text, top_k=5	0.1818	0.1364	0.0785	0.1148

exp02	extracao=PyMuPDF completo (698k/698k chars) em vez de Docling truncado (18%); chunking=auto(sentenca_janela); embedding=nomic-embed-text; top_k=5	0.4091	0.3485	0.2456	0.3133
exp02b	extracao=PyMuPDF completo, chunking=hierarquico FORCADO (igual ao Fase 0, para isolar so o efeito da extracao); embedding=nomic-embed-text; top_k=5	0.3182	0.2879	0.2261	0.2984

A análise comparativa entre os experimentos exp01 e exp02b, que diferem exclusivamente no motor de extração, demonstra que a correção da leitura do documento foi responsável por praticamente dobrar ou triplicar todas as métricas de recuperação.

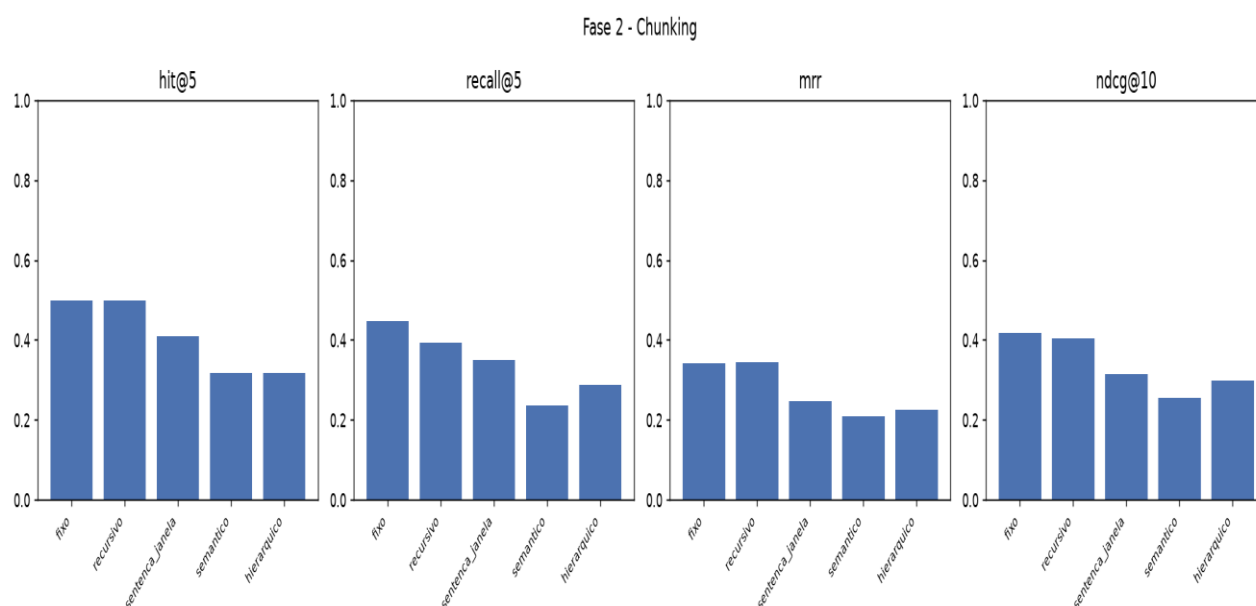
Este achado consolida a principal premissa empírica do estudo: um erro de ingestão comprometeu o Recall em cerca de 80%, configurando uma falha sistêmica invisível aos mecanismos de avaliação do próprio pipeline. O resultado evidencia a necessidade metodológica primária de mensurar a integridade dos dados de entrada antes de otimizar modelos de embedding ou estratégias de particionamento.

3.2. Fase 2: Estratégias de Chunking

Com a extração corrigida, testou-se o impacto de diferentes técnicas de *chunking* (particionamento de texto), mantendo o modelo de *embedding* constante. Os resultados apontaram que o particionamento fixo (200 palavras) superou métodos mais complexos, como o particionamento semântico ou hierárquico.

Tabela 2: Comparativo de métricas de recuperação por técnica de particionamento.

Técnica	Hit@5	Recall@5	MRR	NDCG@10
exp_chunk_fixo	0.5	0.447	0.3415	0.4166
exp_chunk_recurso	0.5	0.3939	0.3438	0.403
exp_chunk_sentenca_janela	0.4091	0.3485	0.2456	0.3133
exp_chunk_hierarquico	0.3182	0.2879	0.2261	0.2984
exp_chunk_semantico	0.3182	0.2348	0.2087	0.256



Comparação das 5 técnicas de chunking (Fase 2).

Os resultados (Tabela 2) demonstram que, contrariando a premissa de que abordagens analíticas mais sofisticadas apresentariam melhor desempenho, o particionamento fixo (200 palavras) obteve as melhores métricas gerais, seguido de perto pelo método recursivo. Em contrapartida, as estratégias semântica e hierárquica registraram os piores índices de recuperação.

Essa superioridade empírica do método de corte determinístico pode ser explicada pela extrema heterogeneidade estrutural do corpus constitucional. Artigos como o 5º possuem extensões massivas — contendo 78 incisos e aproximadamente 14 mil caracteres —, enquanto outros consistem em apenas uma linha. O corte fixo em blocos de 200 palavras atua mitigando essa variância, homogeneizando a densidade informacional dos fragmentos.

Por outro lado, as estratégias baseadas em janelas de sentença fragmentaram excessivamente os artigos longos, gerando múltiplos fragmentos quase duplicados e com baixa informatividade específica. De maneira oposta, as abordagens hierárquica e semântica tenderam a agrupar vários artigos curtos em um mesmo bloco, o que diluiu a especificidade da representação vetorial (embedding) e prejudicou a precisão da etapa de recuperação.

3.3. Fase 3: Avaliação de Modelos de Vetorização (*Embedding*)

Para definir a melhor representação semântica, conduziu-se um *benchmark* comparativo (274 artigos contra 22 perguntas, com \$k=10\$) entre três modelos: o padrão nomic-embed-text (768 dimensões) e os modelos bge-m3 e mxbai-embed-large (ambos com 1024 dimensões).

Tabela 3: Desempenho comparativo dos modelos de embedding.

Modelo	Hit@10	Recall@10	MRR	NDCG@10	AUC	Latência (s)
nomi-embed-text	0.8182	0.7121	0.5278	0.5308	0.9292	22.02
bge-m3	1.0000	0.9848	0.9356	0.9215	0.9930	29.53
mxbai-embed-large	0.8182	0.7197	0.5829	0.5433	0.9589	22.68

O modelo bge-m3 apresentou desempenho significativamente superior em todas as métricas, atingindo um NDCG@10 de 0.9215 na bancada isolada. Quando reindexado no pipeline principal (combinado ao *chunking* fixo), o sistema alcançou um NDCG@10 de 0.7045, representando quase o dobro do melhor resultado da Fase 2.

O desempenho inferior do mxbai-embed-large, apesar de possuir a mesma dimensionalidade do bge-m3, pode ser atribuído à sua limitação de janela de contexto (512 *tokens* contra 8192 do bge-m3), o que força o truncamento de artigos longos, como o Artigo 5º da Constituição.

3.4. Fase 4: Parâmetros de Recuperação e Busca Híbrida

A Fase 4 avaliou o impacto do limite de documentos recuperados (*top_k*) nas métricas finais. Observou-se que o *Recall* sobe de forma consistente com o aumento do *top_k*, subindo de 0.6439 em *k*=3 para 0.8864 em *k*=10. Contudo, esse aumento impõe um custo direto na latência de geração e no consumo de *tokens* pelo LLM. Optou-se por manter o parâmetro ótimo em *k*=5.

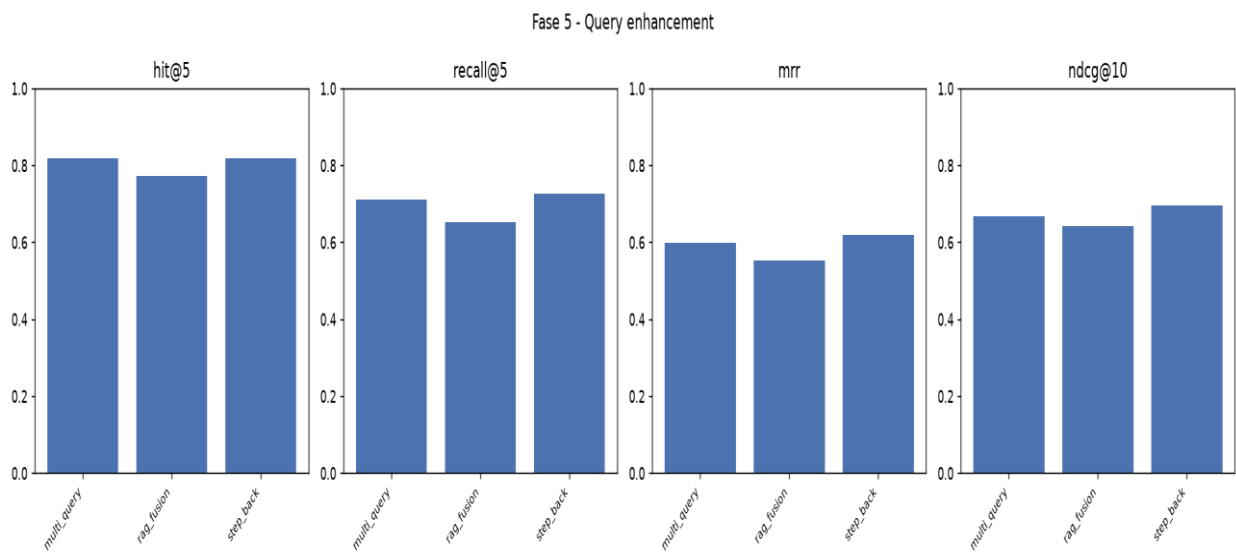
Nesta mesma etapa, introduziu-se a abordagem de busca híbrida, que funde a recuperação lexical (BM25) à busca densa por meio do algoritmo *Reciprocal Rank Fusion* (RRF). Esta configuração alcançou um Hit@5 de 0.8636 e um NDCG@10 de 0.7089, consolidando-se como a arquitetura de recuperação mais eficiente até este estágio do estudo.

3.5. Fase 5: Expansão e Reescrita de Consulta (*Query Enhancement*)

Testou-se o impacto de técnicas de pré-processamento da consulta de entrada para mitigar ambiguidades semânticas. As abordagens avaliadas incluíram *Step-back*, *Multi-query* e *RAG-Fusion*.

Tabela 4: Comparativo de técnicas de expansão de consulta.

Técnica	Hit@5	Recall@5	MRR	NDCG@10
Busca Híbrida (Baseline da Fase 4)	0.8636	0.7348	0.6788	0.7089
Step-back	0.8182	0.7273	0.6201	0.6963
Multi-query	0.8182	0.7121	0.5996	0.6687
RAG-Fusion	0.7727	0.6515	0.5522	0.6408



Comparação das técnicas de query enhancement (Fase 5).

Os resultados (Tabela 4) indicam que nenhuma técnica de reescrita superou o desempenho da busca híbrida simples, tampouco a busca densa pura. Observou-se uma queda generalizada na métrica MRR em todas as três técnicas. A análise sugere que, como o *embedding* de base (bge-m3) já possui forte alinhamento e as perguntas do conjunto de testes são objetivas, o processo de reescrita introduziu variações que dispersaram a ordenação, configurando ruído em vez de expansão útil.

3.6. Fase 6: Reordenação Semântica (*Reranking*)

Para refinar a precisão do topo da lista recuperada, implementou-se um estágio de reordenação (*reranking*). O sistema foi ajustado para recuperar os 20 melhores candidatos via busca densa e, em seguida, submetê-los a um *cross-encoder* (BAAI/bge-reranker-v2-m3) para classificar os 5 documentos finais a serem enviados ao LLM.

Tabela 5: Impacto da etapa de *Reranking* na qualidade e latência.

Configuração	Hit@5	Recall@5	MRR	NDCG@10	Latência Méd. (s)
Busca Densa Simples (Fase 3)	0.8182	0.7273	0.6458	0.7045	2.71
Busca Híbrida (Fase 4)	0.8636	0.7348	0.6788	0.7089	2.78
Rerank (Cross- encoder)	0.9091	0.7879	0.8333	0.8350	16.17

A aplicação do *cross-encoder* proporcionou o incremento mais expressivo de qualidade de toda a avaliação: **o MRR subiu de 0.6788 para 0.8333, e o NDCG@10 saltou de 0.7089 para 0.8350 (Tabela 5)**. No entanto, esse ganho de acurácia gerou um *trade-off* computacional substancial. Devido ao processamento inferencial par a par em CPU (20 interações pergunta-documento por consulta), a latência média escalou drasticamente de 2,78 segundos para 16,17

segundos, evidenciando o custo operacional de arquiteturas de dois estágios (*two-stage retrieval*).

3.7. Fase 7: Recuperação Baseada em Grafos (*LightRAG*)

A sétima fase avaliou a eficácia de uma arquitetura de recuperação baseada em grafos de conhecimento (*LightRAG*). Devido a restrições rigorosas de limite de requisições da API do provedor (cota diária de tokens e limite de requisições por minuto), a ingestão foi delimitada a um subconjunto coeso do corpus: o Título II da Constituição (Art. 5º ao 17), resultando em um grafo com 152 nós e 99 arestas (entidades e relações).

Por limitações de instrumentação da biblioteca utilizada, que não retorna os identificadores originais dos documentos nos metadados, não foi possível aplicar as métricas quantitativas (Hit@K, Recall@K, NDCG@K) sob a mesma metodologia do OpenSearch. Consequentemente, adotou-se uma avaliação qualitativa baseada em perguntas de múltiplos saltos (multi-hop).

A análise qualitativa indicou que, para perguntas contidas estritamente no escopo ingerido, o modelo apresentou excelente capacidade de síntese relacional. No entanto, o sistema demonstrou uma vulnerabilidade crítica de fidelidade (faithfulness) ao lidar com extrapolação de domínio.

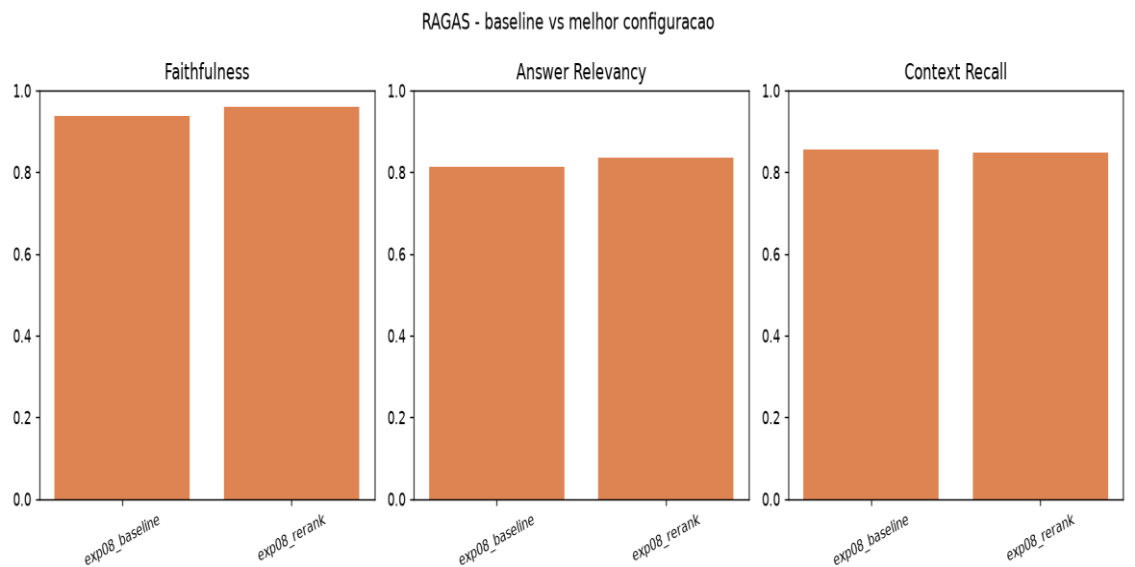
Ao processar consultas dependentes de artigos não indexados no grafo (como o Art. 227), o modelo gerou respostas com alto grau de confiança utilizando conhecimento paramétrico externo (alucinação), atribuindo incorretamente as garantias do Art. 227 aos artigos indexados (Art. 5º, 6º e 7º). Esse comportamento evidencia o risco substancial de imprecisão ao implementar arquiteturas de grafos alimentadas apenas por subconjuntos parciais de um domínio.

3.8. Fase 8: Avaliação de Geração (RAGAS)

A etapa final aplicou o *framework* RAGAS para mensurar a qualidade gerativa ponta a ponta, comparando a busca densa simples (*baseline* otimizado) com a melhor configuração de reordenação (*reranking*). A avaliação utilizou o LLM como juiz (llama-3.3-70b-versatile).

Tabela 6: Comparativo de métricas de geração (RAGAS).

Configuração	Faithfulness	Answer Relevancy	Context Recall	Context Precision
Busca Densa Simples	0.9369	0.8128	0.8560	0.8232
Rerank (Cross- encoder)	0.9588	0.8344	0.8484	0.8153



RAGAS: baseline vs melhor configuração (rerank).

Os dados da Tabela 6 indicam que a inserção do *cross-encoder* eleva a Fidelidade (Faithfulness) e a Relevância da Resposta (Answer Relevancy) em aproximadamente 2,2 pontos percentuais.

Curiosamente, observa-se um decréscimo marginal (~0,8 p.p.) nas métricas de abrangência de contexto (Context Recall e Precision). Esse fenômeno ocorre porque o reranker apenas reordena os mesmos

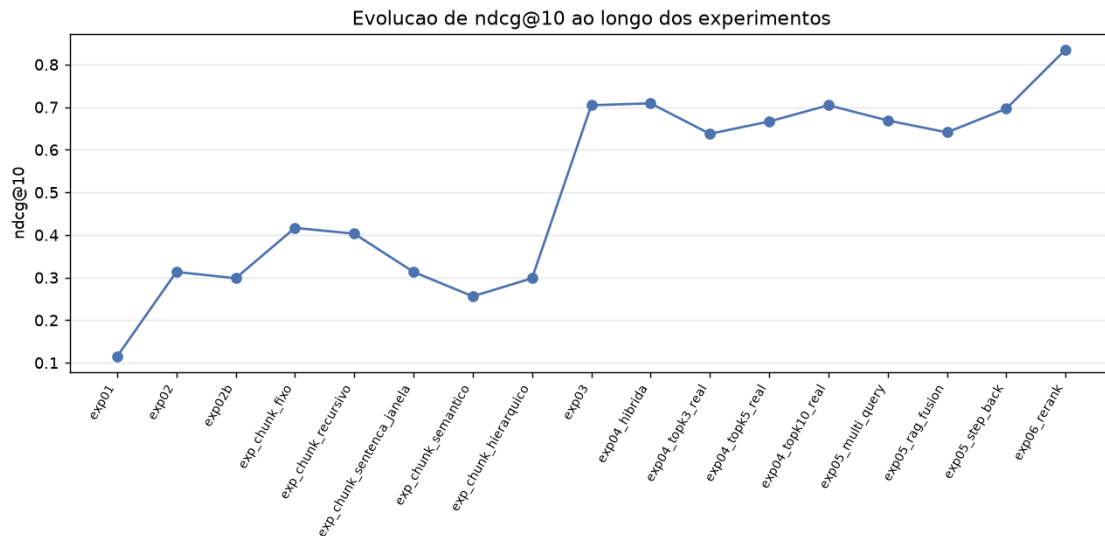
20 candidatos iniciais, priorizando fragmentos textualmente mais focados em detrimento da cobertura bruta. Trata-se de um trade-off válido para este corpus, onde a qualidade percebida da resposta gerada superou a ligeira perda de volume de contexto.

4. Resultados Consolidados e Configuração Ótima

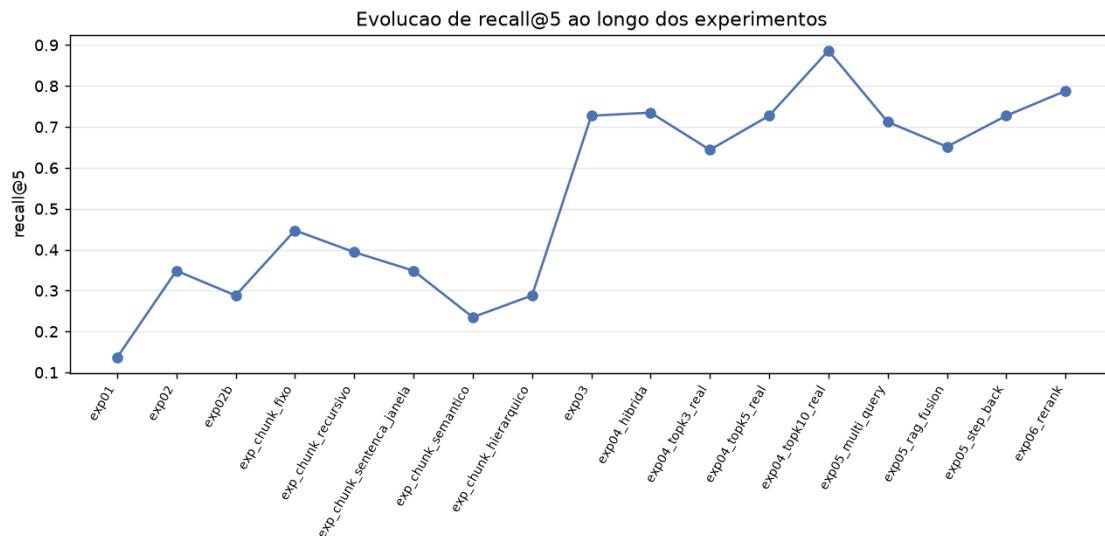
A Tabela 7 consolida a evolução das métricas de recuperação ao longo de todos os experimentos quantitativos, evidenciando o impacto isolado de cada técnica implementada.

Tabela 7: *Evolução consolidada das métricas de recuperação por fase de experimento.*

Fase	Experimento	Hit@5	Recall@5	MRR	NDCG@10
0	Baseline (Docling + Nomic)	0.1818	0.1364	0.0785	0.1148
1	Extração Corrigida (PyMuPDF)	0.4091	0.3485	0.2456	0.3133
2	Chunking Fixo (200 palavras)	0.5000	0.4470	0.3415	0.4166
3	Embedding (bge-m3)	0.8182	0.7273	0.6458	0.7045
4	Busca Híbrida (BM25 + RRF)	0.8636	0.7348	0.6788	0.7089
5	Query Enhancement (Multi-query)	0.8182	0.7121	0.5996	0.6687
6	Reranking (bge-reranker-v2)	0.9091	0.7879	0.8333	0.8350



Evolução do NDCG@10 ao longo da jornada de experimentos.



Evolução do Recall@5.

A partir da análise evolutiva, a arquitetura ótima para o domínio jurídico avaliado foi definida pelas seguintes especificações: extração via **PyMuPDF**, particionamento em **janelas fixas de 200 palavras** (sem sobreposição), vetorização com **bge-m3** (1024 dimensões) e recuperação densa inicial top=20) reordenada por um *cross-encoder* **BAAI/bge-reranker-v2-m3** (top_k final= 5).

Essa composição proporcionou um ganho absoluto expressivo em relação à linha de base: Delta+0.7273 no Hit@5 e Delta+0.7548 no MRR. Os dois maiores saltos individuais de performance decorreram da correção da extração (Fase 1) e da substituição do modelo de *embedding* (Fase 3), caracterizando os ajustes estruturais de maior impacto do projeto.

5. Discussão Crítica e Limitações

A jornada experimental evidenciou importantes compromissos arquiteturais (*trade-offs*) e limitações metodológicas inerentes ao desenvolvimento de sistemas RAG:

5.1. Trade-offs Computacionais e Custos

A etapa de reordenação (*reranking*) entregou a melhor qualidade de ordenação objetiva (MRR/NDCG), porém introduziu um custo operacional severo. A latência aumentou em aproximadamente 6 vezes (de 2,7s para 16,17s por consulta em CPU). Para ambientes de produção, faz-se necessária a adoção de processamento em GPU ou a utilização de *cross-encoders* mais leves.

Em contrapartida, a busca híbrida (BM25+RRF) apresentou a melhor relação custo-benefício, entregando ganhos moderados praticamente sem custo adicional de latência. A abordagem de grafos, por sua vez, demonstrou-se inviável financeiramente para indexação integral sob as restrições de APIs gratuitas devido ao consumo massivo de *tokens*.

5.2. Ineficácia de Estratégias Complexas

Duas hipóteses iniciais foram refutadas ao longo do estudo. Primeiramente, as técnicas de expansão de consulta (*Query Enhancement*) degradaram o desempenho da recuperação, indicando que, diante de consultas objetivas e modelos de *embedding* robustos, a reescrita introduz ruído semântico.

Em segundo lugar, técnicas sofisticadas de particionamento (hierárquico e semântico) apresentaram desempenho inferior ao particionamento linear e fixo. Conclui-se que a heterogeneidade extrema da estrutura jurídica brasileira não se beneficia de heurísticas padrão de agrupamento por tópico.

5.3. Limitações Metodológicas

Os resultados deste estudo devem ser interpretados à luz de suas limitações. O tamanho amostral do conjunto de avaliação (N=22 perguntas) restringe a robustez estatística das análises comparativas entre técnicas de desempenho semelhante (como o *chunking* fixo versus o recursivo). Ademais, a metodologia de validação cruzada (*matching*) entre perguntas e artigos utilizou expressões regulares (*regex*) como *proxy* de correspondência.

Embora funcional, essa abordagem pode gerar falsos positivos ao contabilizar menções incidentais a numerações de artigos em fragmentos que não tratam substancialmente de seu conteúdo.

Por fim, os traçados de observabilidade (LangFuse) foram utilizados para diagnóstico de latência em validações pontuais, não abrangendo sistematicamente todo o conjunto de testes.

6. Conclusão e Trabalhos Futuros

O desenvolvimento da pesquisa reafirmou a premissa fundamental de que a integridade da ingestão de dados precede a otimização de busca. O gargalo inicial do sistema não residia na arquitetura do RAG, mas em uma falha de extração do documento.

Ao final dos experimentos, a configuração empírica ótima para o sistema demonstrou ser composta por: (1) extração via PyMuPDF; (2) particionamento por janelas fixas de 200 palavras; (3) vetorização via **bge-m3**; e (4) busca híbrida reordenada por um *cross-encoder* (**bge-reranker-v2-m3**). Esta combinação elevou a métrica Hit@5 do *baseline* de 0.1818 para 0.9091, e o MRR de 0.0785 para 0.8333.

Os trabalhos futuros devem se concentrar em mitigar a alta latência introduzida pelo **cross-encoder** (avaliando execução em GPU ou a utilização de modelos mais leves), além da expansão quantitativa do **dataset** de avaliação para garantir maior significância estatística na comparação entre as variáveis do pipeline.